

Bug Injection Report

AXI4-Lite Slave IP with Memory

ECE-593 Pre-Si Validation Project — Milestone 5 (5c)

Two bugs were injected to demonstrate that the verification environment detects design and environment failures. Both were reverted after demonstration.

1. RTL Bug — WSTRB Byte-Lane Masking Defeated

Bug Inserted

In `s_axil_wr.sv`, the byte-strobe assignment to memory is overridden to all-ones, so memory writes ignore WSTRB and unconditionally write all four byte lanes.

```
// File: rtl/s_axil_wr.sv

// Original (correct):
assign mem_wstrb = w_strb_lat;

// Bugged:
assign mem_wstrb = '1;
```

Expectation

The scoreboard maintains a shadow-memory model that applies WSTRB on every write. With the DUT now writing all four lanes regardless of WSTRB, the shadow and the DUT will diverge on any partial-strobe write. The scoreboard should detect this on the next read to the same address and report READ MISMATCH (UVM_ERROR).

Actual Response

The scoreboard fired UVM_ERROR on every mismatched read. Representative entries from errors.log:

```
UVM_ERROR [SCB] READ MISMATCH: addr=0x188 got=0x6ce5cd79 expected=0x6c000079
UVM_ERROR [SCB] READ MISMATCH: addr=0x190 got=0xd340aea8 expected=0x8000ae64
UVM_ERROR [SCB] READ MISMATCH: addr=0x320 got=0xdd10ad58 expected=0x801000c8
UVM_ERROR [SCB] READ MISMATCH: addr=0x208 got=0xc9a20456 expected=0x80a20056
...
UVM_ERROR [SCB] RESULT: ** FAIL ** (261 errors)
```

The mismatch signature directly identifies the defect. In the expected value 0x6c000079, the two middle bytes are 0x00 because the test wrote those bytes with WSTRB de-asserted. In the DUT's actual value 0x6ce5cd79, those bytes carry the un-strobed write data, confirming WSTRB was ignored on the write.

Result: bug caught.

2. Environment Bug — Monitor Analysis Port Disconnected

Bug Inserted

In `axil_env.sv` connect_phase, the analysis-port connection between the monitor and the scoreboard is commented out, severing the path between observation and checking.

```
// File: tb/axil_env.sv
```

```
// Original (correct):
    agent.monitor_port.connect(scoreboard.analysis_imp);

// Bugged:
// ENV_BUG: agent.monitor_port.connect(scoreboard.analysis_imp);
```

Expectation

The driver still drives bus traffic, the monitor still observes it, but the scoreboard never receives anything. Its `writes_checked` and `reads_checked` counters should remain zero. The scoreboard's `check_phase` asserts that observed transactions must be non-zero; with both counters at zero it should fire `UVM_ERROR` and the scoreboard report should print FAIL (env check), independent of the errors counter.

Actual Response

`check_phase` fired as expected; the dedicated `errors.log` captured both the env-check error and the resulting FAIL:

```
UVM_ERROR [SCB] ENV CHECK FAILED: scoreboard observed ZERO transactions.
    This indicates the monitor's analysis port is not connected, the agent is
    not active, or the test launched no sequences. Possible env bug - review
    connect_phase wiring.
```

```
UVM_INFO [SCB] Writes checked : 0
UVM_INFO [SCB] Reads checked : 0
UVM_ERROR [SCB] RESULT: ** FAIL ** (env check: zero transactions observed)
```

Note on what this caught: prior to MS5, the scoreboard's PASS/FAIL was based only on a data-error counter; with zero transactions checked, errors was zero, and the scoreboard would have reported PASS — a silent false positive. The `check_phase` guard, added during MS5, converts that silent failure into a loud one.

Result: bug caught.